CS543/ECE549: GANs for Mobile Design

Mark Craft
mscraft2@illinois.edu

Kushagra Tiwary
ktiwary2@illinois.edu

Akhil Alapaty
alapaty2@illinois.edu

Abstract

Currently, mobile app design lacks a system that provide designers with a visual cue to show what their designs may end up looking like. When a user wants to design a mockup of a mobile UI, it's difficult to imagine how the end app using that layout could look like. By using a generative model, such as GANs (Generative Adversarial Networks) and VAEs (Variational Autoencoders), it is possible to show how the mockup could look like by learning the mapping between semantified UIs and how they actually look. Specifically, this problem can be treated like an image-to-image translation problem. We also look at the inverse problem of semantic segmentation of a mobile app UI to produce its semantified counterpart. We treat the semantic segmentation problem as another image-to-image translation problem where we reverse the input and output images.

1 Introduction

GANs are deep neural net architectures that consist of two neural nets set against each other, which explains the "adversarial" nature. GANs have a lot of potential considering their ability to mimic any data distribution. GANs consist of a generator and discriminator network, which together determine whether a test sample matches the initial training data set. After taking in random numbers, the generator returns an image, which is passed on to the discriminator along with images from the training data set. The discriminator then returns probabilities between 0 and 1 for each of these images, as it tries to predict each image's authenticity. Using GANs, we aim to create the generative model that will take in UI semantified screens as inputs and provide the end UI design, as well as the inverse task of semantic segmentation.

2 Related Work

2.1 Generative Models

Recently, generative models based on deep neural nets have been very successful in modeling high-dimensional data (8; 9). Two of the most popular deep generative models are Generative Adversarial Networks (GANs) (4) and Variational Auto-Encoders (VAEs) (10). These models have been shown to have had success in many domains such as image generation, text-to-photo synthesis (27; 18), video generation (23) and speech synthesis (7). They are capable of learning to generate an entire image from a latent random code (17) and conditional versions (14) can take images as input (22). One way to cast our problem is an image-to-image translation problem (i.e., map an image of the current UI to an image of the next component to place next) (6; 29; 26).

2.2 Generative Designs

Mobile UIs are highly structured, with elements placed inside a view hierarchy. To account for this structure, we generate semantic annotations (21) for UI elements and represent the screen as a multi-channel image. From this structured data, we are able to learn design patterns from data (20).

In graphic design, automating layout is a classic problem (5). Using heuristic visual cues and design principles to optimize single-page layouts have shown to be effective (15), and extend well to an interactive tool (16). There has recently been success in extracting semantic structure from documents (25), as well as being able to extend layout generation to generate mobile app UIs (12).

3 Methodology

3.1 Dataset

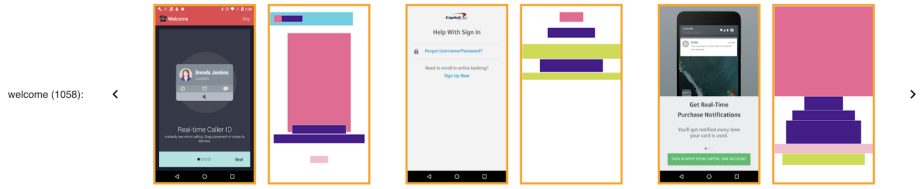

Figure 1: An example set of welcome screens with their semantified counterparts.

As we mentioned in the related work section, we used the semantically annotated UIs from Liu et al. (21) based on the 72k unique UIs from the RICO dataset (3). For both of our proposed tasks, we required pairs of mobile UIs as well as their semantified counterparts. This dataset was perfect for our task as they had these paired images, where each semantic screen could contain 25 types of UI components (or classes).

However, the work presented in Liu et al. attempt to create these semantified images through an iterative, heuristic based approach. Since mobile apps may be designed differently for every developer, their heuristic based approach may not have semantified all of the screens with complete accuracy. Thus, we wanted to show the eficay of our approach on Welcome screens, a subset of screens that appear at the beginning of apps to welcome the user, as seen in Figure 1. As the output of our model is directly effected by the quality of the data we use, after identifying the welcome screens in our dataset, we manually cleaned any screens that were annotated incorrectly.

Although the dataset had 72k unique UIs, since we only use the welcome screens, we only have 1k pairs of UIs to work with. To evaluate our methods, we split our data into a training and validation set, where we use an 80-20 split. Each UI is originally represented via a $128 \times 72 \times 3$ image, but we padded the UIs to a dimension of $128 \times 128 \times 3$ because our deep nets operated on square images.

3.2 Semantified to Mobile UI

The first task that we wanted to solve was the problem of mobile app design lacking a system that provide designers with a visual cue to show what their designs may end up looking like. To solve this problem we can use a generative model to show how a mockup could look like by learning a mapping between semantified UIs and how the UIs actually look like.

The dataset we use provides us with both the semantified UIs as well as the real mobile UI. Thus, we can train a GAN model that is conditioned on the semantified and tries to output a mobile UI. More specifically, our generator takes as input the semantified image, outputs a mobile UI and the discriminator takes in the generated "fake" mobile UI and the ground truth "real" mobile UI. Figure 2 shows examples of input semantified UIs as well as the generated output and ground truth UI.

3.3 Semantic Segmentation of Mobile UIs

In addition to the first task, we could also solve the inverse of this problem; we can do semantic segmentation of the actual UIs to get the semantified UIs. Although Liu et al. attempts to solve this problem, they use a heuristic based approach and is not extendible to all UIs due to relying on information in the UI's view hierarchy which is not consistent throughout apps. Using a generative model that takes in an image as input, it is also possible to perform semantic segmentation.

For this task, we can train a GAN model that is conditioned on the mobile UI and tries to output the semantified UI or the semantic segmentation. More specifically, our generator takes as input the

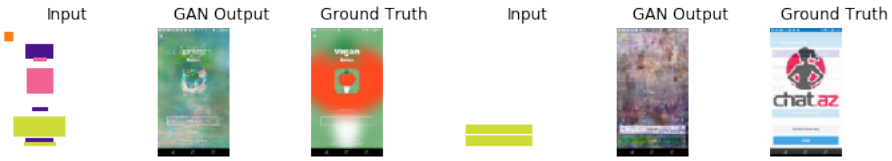

Figure 2: Two example sets of semantified to mobile UI from the test set. The first image is the input to the GAN, the second image is the output from the GAN and the third image is the ground truth segmentation.

mobile UI, outputs the semantic segmentation and the discriminator takes in the generated "fake" semantic segmentation and the ground truth "real" semantic segmentation. Figure 3 shows examples of input mobile UIs as well as the generated output and ground truth semantic segmentation.

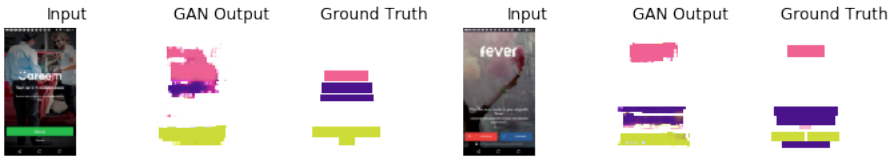

Figure 3: Two example sets of semantic segmentations from the test set. The first image is the input to the GAN, the second image is the output from the GAN and the third image is the ground truth segmentation.

3.4 Objective

GANs optimize a saddle-point objective which pits a parametric generator G against a parametric discriminator D. Both are typically formulated using deep nets and we specify the employed architectures below. The classical minimax objective for GANs can be formulated as follows:

$$\min_G \max_D \mathcal{L}_{GAN}(D, G) := \mathbb{E}_y[\log(D(y))] + \mathbb{E}_x[\log(1 - D(G(x)))]$$

Classical GANs adopt the cross entropy loss function. Following recent works in image-to-image translation (28), we replace this loss with a least squares loss which was shown to stabilizes the training process (13). We use the $a - b$ coding scheme (13) for the discriminator, where a and b are hyper-parameters for the fake data and real data, respectively. The program optimizes the following objectives:

$$\min_D \mathcal{L}_{LSGAN}(D) = \frac{1}{2}\mathbb{E}_y[(D(y) - b)^2] + \frac{1}{2}\mathbb{E}_x[(D(G(x)) - a)^2],$$
$$\min_G \mathcal{L}_{LSGAN}(G) = \frac{1}{2}\mathbb{E}_y[(D(G(x)) - c)^2],$$

where c denotes the value that G wants D to believe for the fake data. Following (13), we choose b as the all-ones vector $\mathbf{1}$, to state that it's real data. Conversely, we choose a as the all-zeros vector to indicate fake data. Finally c is the all-ones vector, encouraging G to fool D. Thus, our final least squares loss is as follows:

$$\min_D \mathcal{L}_{LSGAN}(D) = \frac{1}{2}\mathbb{E}_y[(D(y) - \mathbf{1})^2] + \frac{1}{2}\mathbb{E}_x[(D(G(x)))^2]$$
$$\min_G \mathcal{L}_{LSGAN}(G) = \frac{1}{2}\mathbb{E}_y[(D(G(x)) - \mathbf{1})^2]$$

In addition to the least squares GAN loss $\mathcal{L}_{LSGAN}(G, D)$ we found an additional $\mathcal{L}_{L1}(G)$ loss to be helpful when training the generator. All in all, when training the generator we therefore address the following objective:

$$\min_G \mathcal{L}_{LSGAN}(G) + \lambda \mathcal{L}_{L1}(G),$$

where $\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y}[\|y - G(x)\|_1]$.

For our adversarial loss, we use the mean squared error (MSE) loss combined with an L1 distance loss. We used the L1 loss instead of the L2 loss because the L2 loss is known to produce blurry results (11).

3.5 Architecture

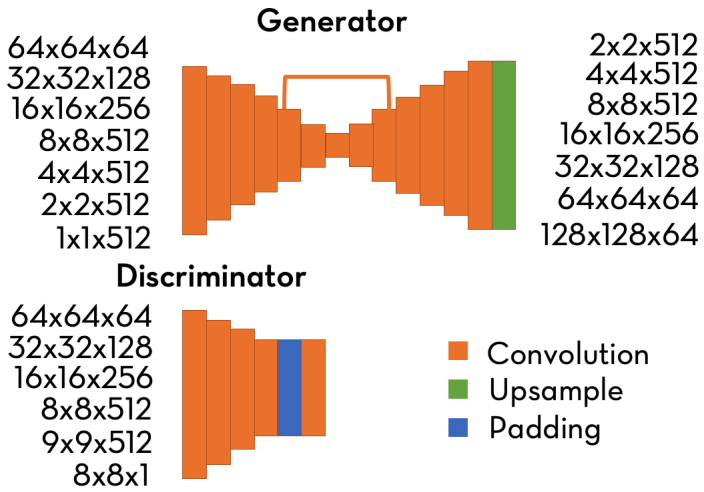

Figure 4: Detailed architecture of the Generator and Discriminator of the GAN model we used.

Our generator and discriminator architectures are based on the architecture presented in Pix2Pix (6).

Specifically, our generator is based on an auto-encoder style "U-Net" architecture. Both the encoder and decoder within the generator use a stack of module-based layers composed of convolution, batch-normalization, and a Leaky Rectified Linear Unit (LeakyReLU) activation function. The encoder consists of 7 modules with a filter size of 4×4 in each of its convolutional layers. Also, normalization is only applied after the first module. The decoder is arranged in the reverse order of the encoder using six transpose convolution layers and ReLU activation functions instead of LeakyReLUs. It is then followed by an upsampling layer, a convolutional layer, and a hyperbolic tangent function to produce a pixel-wise prediction at a resolution of $128 \times 128 \times 3$.

We modified the "U-Net" architecture to reduce skip connections which we found to result in the best trade-off between training time and performance. In particular, we only use one skip connection which links the 5th module of the encoder with the 2nd module of the decoder.

The discriminator $D(y)$ computes the likelihood that its input y is a output originating from the given dataset as opposed to being artificially generated by the generator. To this end we use a convolutional "PatchGAN" classifier consisting of 4 module layers which share the same form as the encoder in our generator. Those are followed by a padding and a convolution layer to reach a patch size of $8 \times 8 \times 1$.

PatchGAN differentiates two images at the scale of image patches. Different patch sizes were assessed (6) and the authors found 16×16 to successfully guide the generator to produce a sufficiently sharp image at a resolution of 256×256. Therefore we decrease the patch size proportionally to the input size that we use (128×128).

A detailed diagram of our GAN architecture is shown in Figure 4.

4 Semantified to Mobile UI Evaluation Metrics

Evaluating synthesized photos is an open and difficult problem. One of the most accurate measurement of image quality is through the use of human judgment, most commonly through the use of Amazon Mechanical Turk. However, we wanted to evaluate the results of our semantified to mobile UI task through some quantitative measures that would not rely on human judgment. We decided to use image quality measures to measure the quality of our generated images; we use structural similarity index (SSIM) (2), multi-scale structural similarity index (MSSSIM) (24), universal image quality index (UQI) (1), and visual information fidelity (VIF) (19). For each of these measures, they were evaluated on the test set on the pair of ground truth UIs and generated UIs.

4.1 Structural Similarity Index

The structural similarity index (SSIM) serves to discount features of images that are not relevant to perception. This is done using luminance (I), contrast (C), and structure (S), which are computed as shown below:

$$I(x,y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1} \qquad C(x,y) = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2} \qquad S(x,y) = \frac{\sigma_{xy} + C_3}{\sigma_x\sigma_y + C_3}$$

The variables $\mu_x, \mu_y, \sigma_x, \sigma_y$ refer to the mean and standard deviation of pixel intensity in an image region centered at either the x or y coordinate. The variable σ_{xy} refers to the correlation coefficient between complementing pixels in the mentioned image regions. The other constants in the equations are merely included for stability purposes. Using those three values, the SSIM output is computed as follows:

$$SSIM(x,y) = I(x,y)^\alpha C(x,y)^\beta S(x,y)^\gamma$$

4.2 Multi-Scale Structural Similarity Index

The SSIM is considered at multiple scales, which is then known as the Multi-Scale Structural Similarity Index (MS-SSIM). Input images x and y are filtered by a low-pass filter and downsampled by a factor of 2. Since this happens iteratively, the factor $2^(j-1)$ refers to the downsampling factor, where j represents the initial image being downsampled. The contrast and structure components are applied and weighed at each scale, while the luminance component is applied only to the last scale, which is labeled as M, as seen below:

$$MM - SSIM(x,y) = I_M(x,y)^{\alpha_M} \prod_{j=1}^{M} C(x,y)^{\beta_j} S(x,y)^{\gamma_j}$$

MM-SSIM output values fall between the range of 0 and 1, with 0 indicating low similarity and 1 indicating high similarity.

4.3 Universal Image Quality Index

As opposed to using age-old error computation techniques, a newly suggested index is derived from utilizing the following three factors: luminance distortion, contrast distortion, and loss of correlation. With x and y respectively being the original and test image signals, the Universal Image Quality Index can be defined as follows:

$$Q = \frac{4\sigma_x\sigma_y\bar{x}\bar{y}}{(\sigma_x^2 + \sigma_y^2)[(\bar{x}^2) + (\bar{y}^2)]}$$

4.4 Visual Information Fidelity

The Visual Information Fidelity (VIF) measure is essentially the proportion of the reference image information that could be extracted from the test signal. Once this is done over all channels of interest,

we receive the following equation for VIF, where E represents the reference and F represents the test:

$$VIF = \frac{\sum_{j \in channels} I(\vec{C}^{(}N,j); \vec{F}^{(}N,j)|s^{(}N,j)}{\sum_{j \in channels} I(\vec{C}^{(}N,j); \vec{E}^{(}N,j)|s^{(}N,j)}$$

Table 1: Results of semantified to mobile UI task on the Mobile App Dataset.

SSIM	MSSSIM	UQI	VIF
99.9	99.9	84.8	2.2

5 Semantic Segmentation Evaluation Metrics

For our semantic segmentation task, we report four metrics from common semantic segmentation and scene parsing evaluations that are variations on pixel accuracy and region intersection over union (IOU).

We define the four metrics as follows. Let n_{ij} be the number of pixels of class i predicted to belong to class j, where there are n_{cl} different classes, and let $t_i = \sum_j n_{ij}$ be the total number of pixels of class i. We compute:

- Pixel Accuracy: $\frac{\sum_i n_{ii}}{\sum_i t_i}$
- Mean Accuracy: $\frac{1}{n_{cl}} \sum_i \frac{n_{ii}}{t_i}$
- Mean IOU: $\frac{1}{n_{cl}} \sum_i \frac{n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}}$
- Frequency Weighted IOU: $(\sum_k t_k)^{-1} \sum_i \frac{t_i n_{ii}}{t_i + \sum_j n_{ji} - n_{ii}}$

5.1 Post-processing

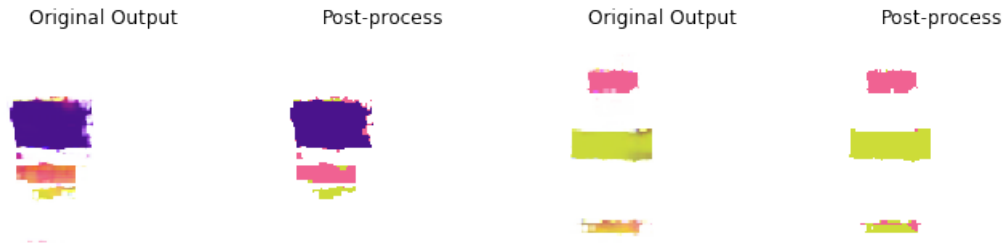

Figure 5: Two sets of original outputs from the GAN and their post-processed counterparts.

Prior to computing our evaluation metrics for the semantic segmentation task, we needed to perform some post-processing. As seen in Figure 3, the output of the semantic segmentation task have some blurred edges as well as a mix of colors. This output cannot be directly used because for some of the pixels, it may be ambiguous as to what class they are predicting for or the exact region that they are predicting. Thus for every pixel in the predicted segmentation, we use euclidean distance to see which of the 25 classes or background that the pixel is closest to. Then, we replace the color with the closest valid color as shown in Figure 5. After doing this post-processing step, we can then apply our evaluation metrics.

5.2 IOU

IOU is an evaluation metric that reports the area of the overlap between the prediction and ground truth and dividing by the area of union (the area encompassed by both the prediction and ground truth). Note that IOU is a harsh metric, particularly because it only gives credit for the area of overlap and heavily penalizes for any area that is not overlapping. In the case where the prediction is offset from the ground truth the score may be very low. We use two variants of the IOU metric as defined above, mean IOU and frequency weighted IOU.

5.3 Accuracy

Pixel accuracy is an alternative metric beyond IOU which is also often used in semantic segmentation tasks. This metric simply reports the fraction of pixels in the image which are correctly classified.

Pixel accuracy is less harsh than IOU but it may be misleading: in cases where the ground truth component is small, the metric is biased to mainly report how well background is identified rather than focusing on the components and classes of interest. We use two variants of the Accuracy metric as defined above, pixel accuracy and mean accuracy.

Table 2: Results of semantic segmentation on the Mobile App Dataset.

Pixel Acc.	Mean acc.	Mean IOU	F.W. IOU
71.4	49.7	17.2	60.1

6 Discussion and Conclusions

For this project, we had two tasks with regards to our mobile app dataset. In our first task of semantified UIs to mobile UIs, we can see that qualitatively, it is doing some reasonable things; for example, the model is able to understand that text buttons should have some sort of clearly defined border. However, as a semantified UI may have many possible mobile UIs corresponding to it, deciding whether the output is good is more subjective. To try and get some quantitative measures, we used image quality measurements. Although these methods have been used and proven to work well for many image tasks, it may not have completely transferred well to our task; our images do not consist of "natural" images so they have qualities that would be misleading for some of these measures.

In addition, although we were able to get some good results, our model is not able to generate that much variation; the Pix2pix based models tends to ignore the latent z code used in GANs and result in an almost deterministic mapping. Although we tried to train another model based on the BicycleGAN (29) architecture and objective, we were not able to get as comparable of results.

For our second task of semantic segmentation, we see that qualitatively and quantitatively, it did pretty well. The model was able to segment the text and text button classes fairly accurately. However, the model had a tough time dealing with segmenting the images in the mobile UIs; in a mobile app, it can be ambiguous it something is text or an image of a text, or if something is an image or a background image. Due to the ambiguities in how these visually look, the model had some difficulties.

For future work, it would be interesting to see if an approach could be applied that utilized the fact that every component (class) in the UI is rectangular; the GAN model generated segmentations that were roughly rectangular but could have been improved. In addition, a multi-modal apporach may be possible, where a model is both trained on the raw image pixels as well as the view hierarchy that represents the structure of the app as done in Liu et al.

7 Statement of Individual Contribution

Our group regularly met on a weekly basis while engaging daily on our group chat in order to ensure timely progress. Each of us was responsible for cleaning the data set, given that there are some entries that are incorrectly labelled, which would inadvertently affect the GAN performance. Individually, Kushagra and Akhil designed the initial GAN, before then modifying it in order to reach certain

defined accuracy thresholds. Mark, with his vast expertise in deep learning, made several strides in solving the semantic segmentation problem by taking the responsibility to train the GAN on the UI Welcome screens from the data set. Additionally, Mark completed the post-processing of the output because in some cases, there was ambiguity in terms of the pixel predictions. Once the true predictions were finalized following euclidean distance computations, the results were passed back to Kushagra and Akhil, who were then responsible for evaluating these results. Using the IOU and Pixel accuracy metrics, they were able to present final results.

References

[1] and A. C. Bovik. 2002. A universal image quality index. *IEEE Signal Processing Letters* 9, 3 (March 2002), 81–84. DOI:http://dx.doi.org/10.1109/97.995823

[2] and A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (April 2004), 600–612. DOI:http://dx.doi.org/10.1109/TIP.2003.819861

[3] Chad Franzen Joshua Hibschman Daniel Afergan Yang Li Jeffrey Nichols Biplab Deka, Zifeng Huang and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. *UIST* (2017).

[4] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Networks. (2014).

[5] Nathan Hurst, Wilmot Li, and Kim Marriott. 2009. Review of Automatic Document Formatting. In *Proceedings of the 9th ACM Symposium on Document Engineering (DocEng '09)*. ACM, New York, NY, USA, 99–108. DOI:http://dx.doi.org/10.1145/1600193.1600217

[6] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. 2016. Image-to-Image Translation with Conditional Adversarial Networks. *CoRR* abs/1611.07004 (2016). http://arxiv.org/abs/1611.07004

[7] Lauri Juvela, Bajibabu Bollepalli, Junichi Yamagishi, and Paavo Alku. 2018. Waveform generation for text-to-speech synthesis using pitch-synchronous multi-scale generative adversarial networks. (2018).

[8] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. 2017. Progressive Growing of GANs for Improved Quality, Stability, and Variation. (2017).

[9] Tero Karras, Samuli Laine, and Timo Aila. 2018. A Style-Based Generator Architecture for Generative Adversarial Networks. (2018).

[10] Diederik P Kingma and Max Welling. 2013. Auto-Encoding Variational Bayes. (2013).

[11] Anders Boesen Lindbo Larsen, Søren Kaae Sønderby, Hugo Larochelle, and Ole Winther. 2015. Autoencoding beyond pixels using a learned similarity metric. (2015).

[12] Jianan Li, Jimei Yang, Aaron Hertzmann, Jianming Zhang, and Tingfa Xu. 2019. LayoutGAN: Generating Graphic Layouts with Wireframe Discriminators. (2019).

[13] Xudong Mao, Qing Li, Haoran Xie, Raymond Y. K. Lau, Zhen Wang, and Stephen Paul Smolley. 2016. Least Squares Generative Adversarial Networks. (2016).

[14] Mehdi Mirza and Simon Osindero. 2014. Conditional Generative Adversarial Nets. (2014).

[15] Peter ODonovan, Aseem Agarwala, and Aaron Hertzmann. 2014. Learning Layouts for Single-PageGraphic Designs. *IEEE Transactions on Visualization and Computer Graphics* 20, 8 (Aug. 2014), 1200–1213. DOI:http://dx.doi.org/10.1109/TVCG.2014.48

[16] Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. 2015. DesignScape: Design with Interactive Layout Suggestions. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. ACM, New York, NY, USA, 1221–1224. DOI:http://dx.doi.org/10.1145/2702123.2702149

[17] Alec Radford, Luke Metz, and Soumith Chintala. 2015. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. (2015).

[18] Scott Reed, Zeynep Akata, Xinchen Yan, Lajanugen Logeswaran, Bernt Schiele, and Honglak Lee. 2016. Generative Adversarial Text to Image Synthesis. (2016).

[19] H. R. Sheikh and A. C. Bovik. 2006. Image information and visual quality. *IEEE Transactions on Image Processing* 15, 2 (Feb 2006), 430–444. DOI:http://dx.doi.org/10.1109/TIP.2005.859378

[20] Jerry Talton, Lingfeng Yang, Ranjitha Kumar, Maxine Lim, Noah Goodman, and Radomír Měch. 2012. Learning Design Patterns with Bayesian Grammar Induction. In *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology (UIST '12)*. ACM, New York, NY, USA, 63–74. DOI:http://dx.doi.org/10.1145/2380116.2380127

[21] Jason Situ Ersin Yumer Radomir Mech Thomas F. Liu, Mark Craft and Ranjitha Kumar. 2018. Learning Design Semantics for Mobile Apps. *UIST* (2018).

[22] Aaron van den Oord, Nal Kalchbrenner, Lasse Espeholt, koray kavukcuoglu, Oriol Vinyals, and Alex Graves. 2016. Conditional Image Generation with PixelCNN Decoders. In *Advances in Neural Information Processing Systems 29*, D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett (Eds.). Curran Associates, Inc., 4790–4798. http://papers.nips.cc/paper/6527-conditional-image-generation-with-pixelcnn-decoders.pdf

[23] Carl Vondrick, Hamed Pirsiavash, and Antonio Torralba. 2016. Generating Videos with Scene Dynamics. In *Advances in Neural Information Processing Systems 29*, D. D. Lee, M. Sugiyama, U. V. Luxburg, I. Guyon, and R. Garnett (Eds.). Curran Associates, Inc., 613–621. http://papers.nips.cc/paper/6194-generating-videos-with-scene-dynamics.pdf

[24] Z. Wang, E. P. Simoncelli, and A. C. Bovik. 2003. Multiscale structural similarity for image quality assessment. In *The Thrity-Seventh Asilomar Conference on Signals, Systems Computers, 2003*, Vol. 2. 1398–1402 Vol.2. DOI:http://dx.doi.org/10.1109/ACSSC.2003.1292216

[25] Xiao Yang, Ersin Yumer, Paul Asente, Mike Kraley, Daniel Kifer, and C. Lee Giles. 2017. Learning to Extract Semantic Structure from Documents Using Multimodal Fully Convolutional Neural Network. (2017).

[26] Zili Yi, Hao Zhang, Ping Tan, and Minglun Gong. 2017. DualGAN: Unsupervised Dual Learning for Image-To-Image Translation. In *The IEEE International Conference on Computer Vision (ICCV)*.

[27] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris N. Metaxas. 2017. StackGAN: Text to Photo-Realistic Image Synthesis With Stacked Generative Adversarial Networks. In *The IEEE International Conference on Computer Vision (ICCV)*.

[28] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. 2017a. Unpaired Image-To-Image Translation Using Cycle-Consistent Adversarial Networks. In *The IEEE International Conference on Computer Vision (ICCV)*.

[29] Jun-Yan Zhu, Richard Zhang, Deepak Pathak, Trevor Darrell, Alexei A. Efros, Oliver Wang, and Eli Shechtman. 2017b. Toward Multimodal Image-to-Image Translation. (2017).